

Titanic Survival Prediction

A beginner machine learning project — built step by step using Python, Pandas, and scikit-learn

Project Summary

This project predicts whether a Titanic passenger survived or died, based on features like age, sex, ticket class, fare, and family size. It was built from scratch as a first hands-on machine learning project, using the classic Titanic dataset. The final model — a Logistic Regression classifier — achieves approximately **73.7% accuracy** on unseen test data.

Tools & Libraries Used

- **Python** — programming language
- **Google Colab** — notebook environment (no installation needed)
- **Pandas** — loading, cleaning, and exploring tabular data
- **NumPy** — numerical operations
- **scikit-learn** — model training, splitting data, and evaluation

Dataset

The classic Titanic dataset (891 passengers), containing details such as passenger class, sex, age, number of siblings/spouses aboard, number of parents/children aboard, fare paid, and port of embarkation. Publicly available via Kaggle's "Titanic - Machine Learning from Disaster" competition, or directly from GitHub-hosted mirrors.

Step 1 — Load and Explore the Data

The dataset was loaded into a Pandas DataFrame and inspected using `.head()` and `.info()` to understand its structure: 891 rows, 12 columns, and to spot missing values.

```
import pandas as pd
df = pd.read_csv('train.csv')

print(df.head())
df.info()
```

Finding: Age had 177 missing values, Cabin had 687 missing values, and Embarked had 2 missing values out of 891 total rows.

Step 2 — Explore Survival Patterns (EDA)

Before touching the data, survival rates were explored across key groups using `.groupby()` and `.mean()`, to see which features seemed to matter most.

```
df.groupby('Sex')['Survived'].mean()
df.groupby('Pclass')['Survived'].mean()
df.groupby(['Sex', 'Pclass'])['Survived'].mean()
```

Group	Survival Rate
Female	74.2%
Male	18.9%
1st Class	63.0%
2nd Class	47.3%
3rd Class	24.2%
Female + 1st Class	96.8%
Male + 3rd Class	13.5%

Key insight: Sex was the single strongest predictor of survival, followed by Passenger Class (Pclass). Together, they created dramatic extremes — a 1st-class woman had roughly a 97% survival rate, while a 3rd-class man had only about 13.5%.

Step 3 — Clean Missing Data

- **Age** (177 missing) — filled with the column's median value.
- **Cabin** (687 missing, ~77% of the column) — dropped entirely; too sparse to fill reliably.
- **Embarked** (2 missing) — filled with the column's mode (most frequent port).

```
df['Age'] = df['Age'].fillna(df['Age'].median())
df = df.drop('Cabin', axis=1)
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
```

Step 4 — Drop Unhelpful Columns

Name, Ticket, and PassengerId were removed — these are unique identifiers or raw text with no direct numeric meaning a simple model can use.

```
df = df.drop(['Name', 'Ticket', 'PassengerId'], axis=1)
```

Step 5 — Encode Categorical Columns

Sex (2 categories, binary) was mapped directly to numbers: male = 0, female = 1.

```
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
```

Embarked (3 categories, no natural order) was one-hot encoded instead of mapped to 0/1/2, to avoid implying a false ranking between ports.

```
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True)
```

Result: a fully numeric dataset with 9 columns — Survived, Pclass, Sex, Age, SibSp, Parch, Fare, Embarked_Q, Embarked_S — and zero missing values.

Step 6 — Split Features (X) and Target (y)

The target column (Survived — the answer to predict) was separated from the feature columns (the clues the model is allowed to use).

```
X = df.drop('Survived', axis=1)
y = df['Survived']
```

Step 7 — Train / Test Split

The data was split into a training set (80%) and a test set (20%), so the model could later be evaluated on passengers it never saw during training.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=43
)
```

Result: 712 training rows, 179 test rows.

Step 8 — Train the Model

A Logistic Regression classifier was used — a simple, interpretable model well suited for binary (yes/no) prediction problems like this one.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

Step 9 — Evaluate the Model

Predictions were generated on the held-out test set, then compared against the real answers to measure accuracy.

```
predictions = model.predict(X_test)

from sklearn.metrics import accuracy_score
accuracy_score(y_test, predictions)
```

Metric	Result
Accuracy	73.7%
Correct predictions	~132 / 179

For comparison, always predicting "died" (the majority class) would score about 61-62% accuracy. A 73.7% score means the model genuinely learned meaningful patterns from the data, beyond just guessing

the most common outcome.

Full Pipeline Recap

- Loaded 891 passenger records with Pandas
- Explored survival patterns by Sex, Pclass, and their combination
- Cleaned missing values in Age, Cabin, and Embarked
- Dropped identifier / free-text columns (Name, Ticket, PassengerId)
- Encoded Sex (binary mapping) and Embarked (one-hot encoding)
- Split data into features (X) and target (y)
- Split into training (80%) and testing (20%) sets
- Trained a Logistic Regression model
- Evaluated on unseen test data — achieved 73.7% accuracy

Possible Next Steps

- Try a stronger model, e.g. **RandomForestClassifier**, using the same fit/predict pattern
- Inspect a **confusion matrix** to see exactly what kinds of mistakes the model makes
- Engineer new features — e.g. FamilySize (SibSp + Parch), or extracting Title (Mr/Mrs/Miss) from Name
- Try different values of test_size and random_state to see how stable the accuracy is
- Tune model hyperparameters for further improvement

Built as a self-guided learning project using Python, Pandas, and scikit-learn.